

ROYAUME DU MAROC

Ministere de l'Enseignement Superieur

Sciences de l'Ingenieur - Genie Informatique

PROJET DE FIN D'ETUDES

MyHeath

Plateforme de telemedecine et sante connectee

ASP.NET Core 8 · React · MongoDB · Docker

Realise par : Nezha Fekoussa

Annee universitaire 2025 - 2026

Document a destination du jury / encadrant

Resume / Abstract

Resume (FR)

MyHeath est une plateforme de telemedecine (sante feminine et masculine) avec suivi quotidien, diabete, cycle, messagerie chiffree, coach IA et administration. Ce rapport presente le backend ASP.NET Core 8 realise par Nezha Fekoussa, avec React et MongoDB, Docker et deployment cloud.

Abstract (EN)

MyHeath is a telemedicine platform for women's and men's health tracking, diabetes care, encrypted messaging, AI coaching and admin subscriptions. This report documents the ASP.NET Core 8 backend authored by Nezha Fekoussa, keeping React and MongoDB, with Docker and cloud deployment guidance.

Keywords / Mots-cles

ASP.NET Core 8, React, MongoDB, Docker, JWT, AES-256, telemedicine, Claude AI

Table des matieres

1. Introduction et contexte	5
1.1 Problematique	5
1.2 Objectifs du projet	5
1.3 Pourquoi ASP.NET Core	5
2. Analyse des besoins	6
2.1 Acteurs	6
2.2 Exigences fonctionnelles	6
2.3 Exigences non fonctionnelles	6
3. Architecture technique	7
3.1 Vue d'ensemble	7
3.2 Stack	7
3.3 Structure projet	7
3.4 Compatibilite Node	8
4. Conception detaillee	9
4.1 Modele MongoDB	9
4.2 AES-256-CBC	9
4.3 Analyse predictive	10
5. Realisation backend .NET	11
5.1 Authentification	11
5.2 Modules	11
5.3 Enhancements	12
6. Frontend React	15
7. Docker et deploiement	16
7.1 Conteneurisation	16
7.2 Variables d'environnement	16
7.3 Cloud	16
8. Tests et comptes demo	17
8.1 Scenarios	17
9. Comparaison Node.js vs ASP.NET Core	18
10. Securite	19

11. Conclusion	20
11.1 Livrables	20
Annexe A - Requetes	21
Annexe B - Checklist Docker	22
Annexe C - Glossaire	23

1. Introduction et contexte

1.1 Problematique

L'accès à un suivi de santé continu reste fragmenté. MyHeath centralise le parcours patient-medecin-administrateur autour d'une API sécurisée et d'une interface dashboard.

1.2 Objectifs du projet

- API REST professionnelle en ASP.NET Core 8
- Frontend React conserve (contrat /api compatible)
- Persistance MongoDB + chiffrement AES-256-CBC + JWT
- RDV, rappels médicaments, notifications, audit admin
- Docker et déploiement cloud documentés

1.3 Pourquoi ASP.NET Core

ASP.NET Core 8 s'aligne sur les compétences C# / Web API / DI / JWT. Le projet est réalisé par Nezha Fekoussa.

2. Analyse des besoins

2.1 Acteurs

Acteur	Responsabilites principales
Patient	Suivi, diabete, IA, RDV, medicam
Medecin	Consultations, RDV, messagerie
Administrateur	Users, plans, abonnements, audit

2.2 Exigences fonctionnelles

- Auth + RBAC patient | doctor | admin
- Suivi quotidien + score 0-100, periodes, diabete
- Dossier et chat chiffres, AI Coach Claude
- Abonnements FREE / CARE / PREMIUM

2.3 Exigences non fonctionnelles

- Securite JWT / BCrypt / AES, Docker, Swagger

Figure 2.3 — Diagramme de cas d'utilisation

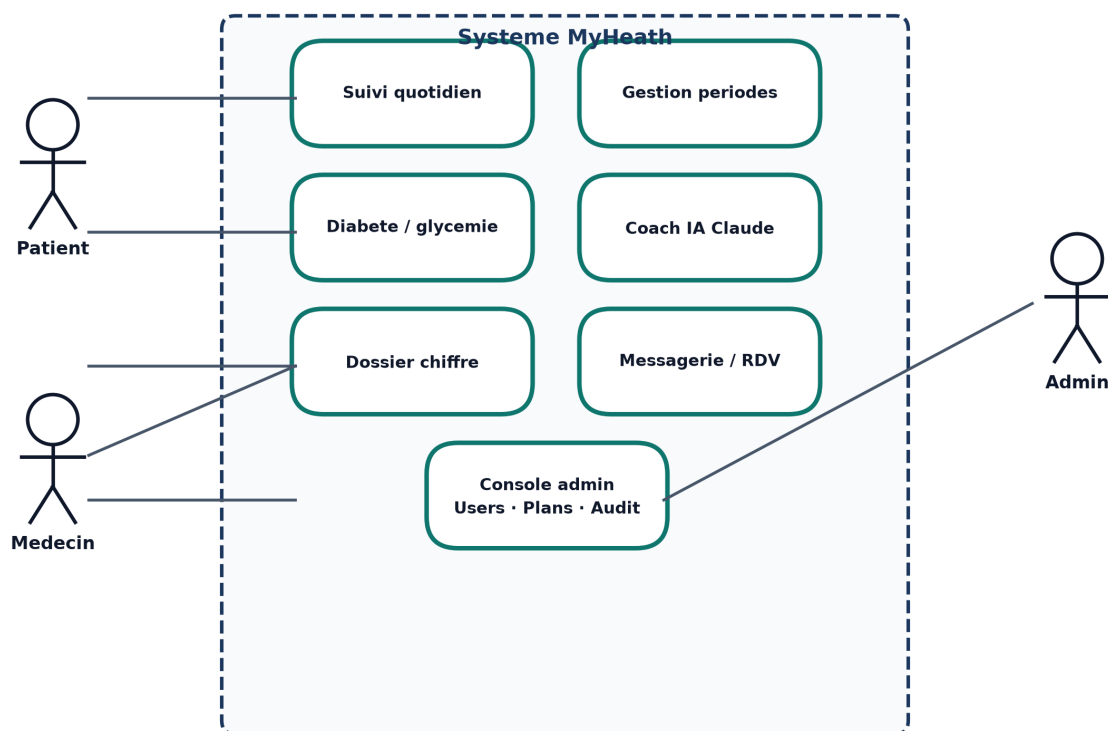


Figure 2.3 - Diagramme de cas d'utilisation

3. Architecture technique

3.1 Vue d'ensemble

Architecture 3-tiers: React -> ASP.NET Core -> MongoDB (VITE_API_URL).

Figure 2.1 — Architecture 3-tiers MyHeath (.NET + React + MongoDB)

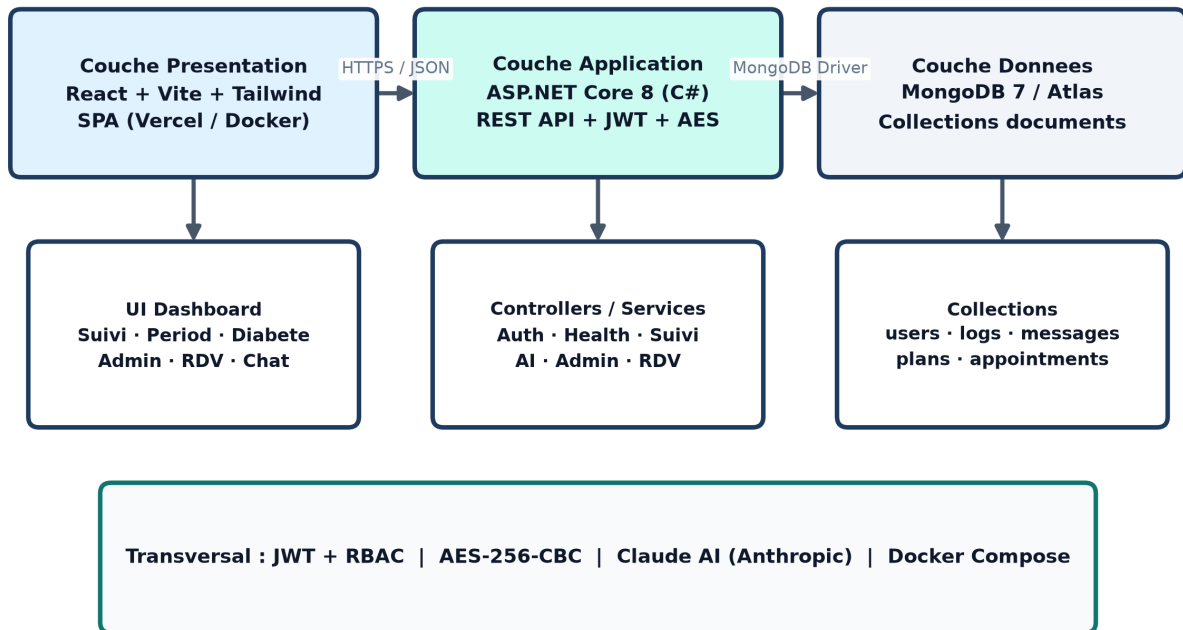


Figure 2.1 - Architecture 3-tiers MyHeath

3.2 Stack

Couche	Technologies
Backend	ASP.NET Core 8, C#, MongoDB.Driv
Frontend	React 18, Vite, Tailwind, Axios
Donnees	MongoDB 7 / Atlas
Ops	Docker Compose, Vercel, Azure

3.3 Structure projet

```

backend-dotnet/MyHeath.Api/
  Controllers/ Services/ Models/ Program.cs
docker-compose.dotnet.yml
  
```

3.4 Compatibilite Node

Meme contrat /api/* ; bascule Node:5000 ou .NET:5080 via VITE_API_URL.

4. Conception detaillee

4.1 Modele MongoDB

- users, healthrecords, symptomlogs, dailyhealthlogs, messages
- subscriptionplans, appointments, notifications, auditlogs

Figure 2.6 — Modele de domaines (extrait)

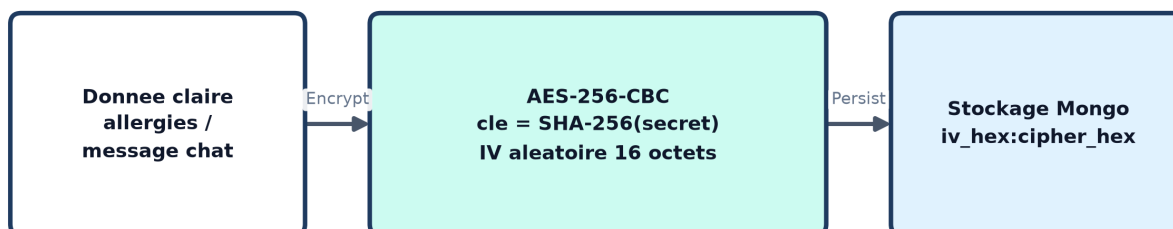


Figure 2.6 - Modele de domaines

4.2 AES-256-CBC

Champs sensibles et messages chiffrés (iv_hex:ciphertext_hex, cle SHA-256).

Figure 2.7 — Chiffrement AES-256-CBC des donnees sensibles



Dechiffrement uniquement cote API apres JWT + RBAC

Figure 2.7 - Chiffrement AES-256-CBC

4.3 Analyse predictive

HealthScoreService (0-100) et CycleAnalyzer (phase, ovulation, anomalies).

Figure 2.8 — Pipeline score sante & analyse de cycle

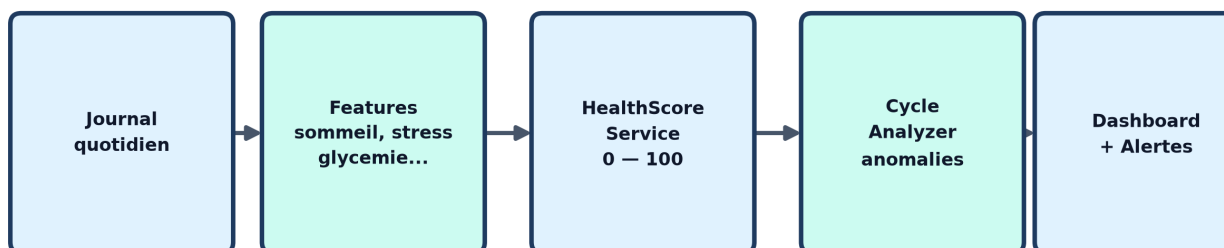


Figure 2.8 - Pipeline predictif

5. Realisation backend .NET

5.1 Authentication

BCrypt cost 12, JWT HMAC-SHA256 7 jours, [Authorize(Roles=...)].

Figure 2.4 — Sequence : authentication JWT

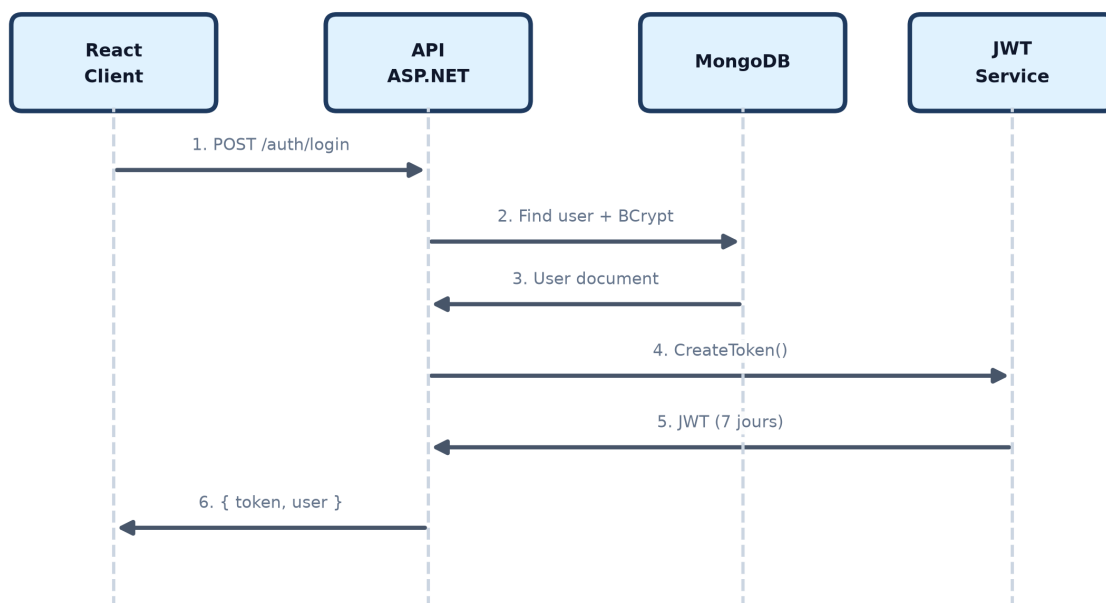
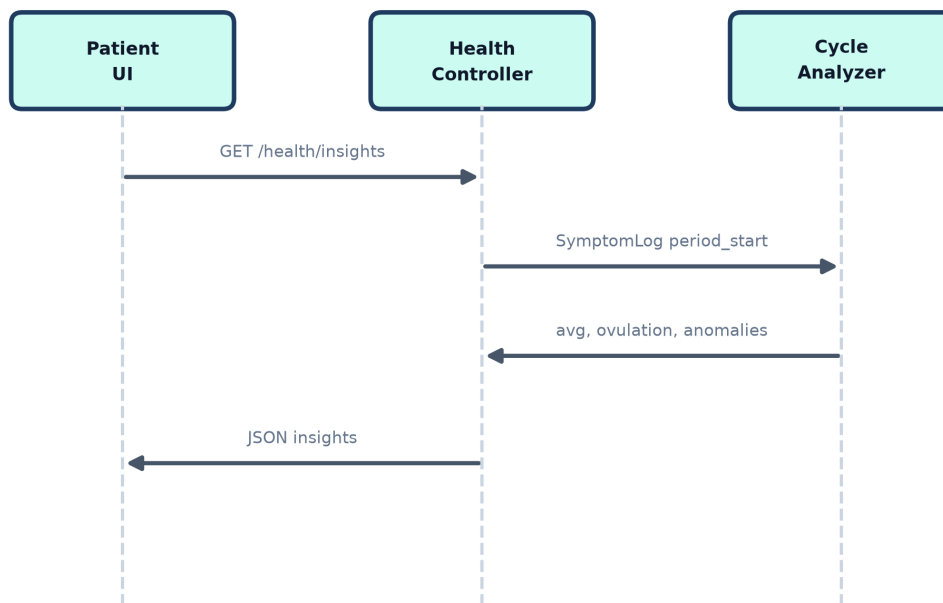


Figure 2.4 - Sequence authentication

5.2 Modules

- Health, Suivi, Chat, AI, Admin, Appointments, Medications, Notifications

Figure 2.5 — Sequence : calcul des insights cycle*Figure 2.5 - Sequence insights*

5.3 Enhancements

Fonctionnalite	Endpoint
Rendez-vous	/api/appointments
Notifications	/api/notifications
Medicaments	/api/medications
Audit	/api/admin/audit
Swagger	/swagger

Figure 3.1 — Organisation du backend ASP.NET Core

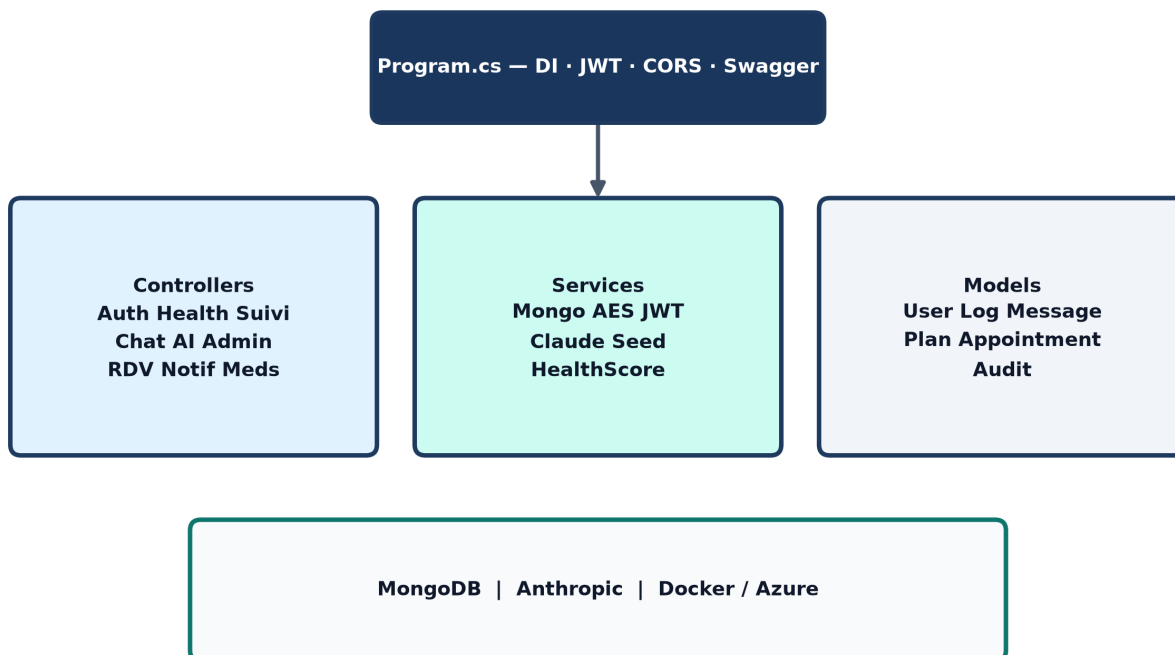


Figure 3.1 - Organisation backend

Figure 3.3 — Cartographie des routes API (/api)

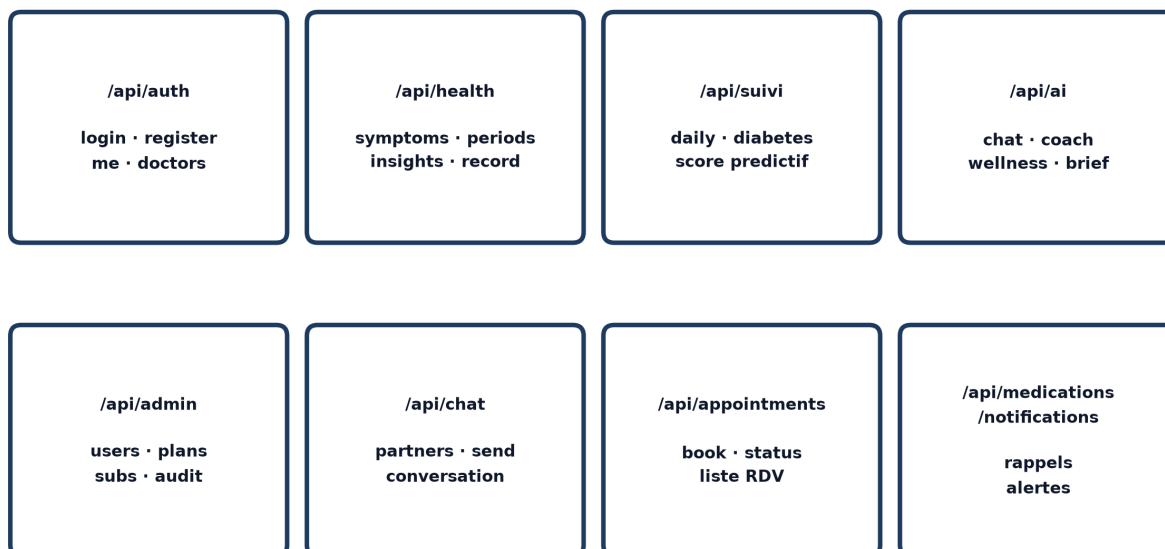
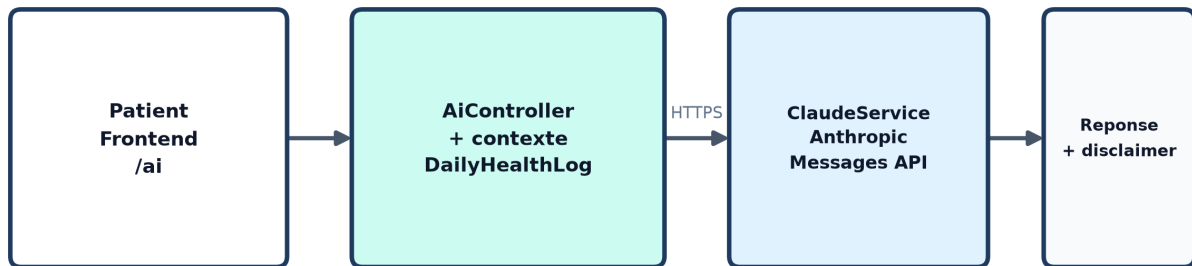


Figure 3.3 - Routes API

Figure 2.9 — Integration MyHeath AI Coach (Claude)



Usage educatif uniquement — pas un diagnostic medical

Figure 2.9 - AI Coach Claude

6. Frontend React

Dashboard sidebar: Admin, Suivi, Period, Diabetes, Medications, AI, RDV, Chat.

- Auteur du livrable: Nezha Fekoussa

7. Docker et deployment

7.1 Conteneurisation

```
docker compose -f docker-compose.dotnet.yml up --build
# API http://localhost:5080/api/healthcheck
# UI http://localhost:5173
```

Figure 2.2 — Topologie de deployment (Docker / Cloud)

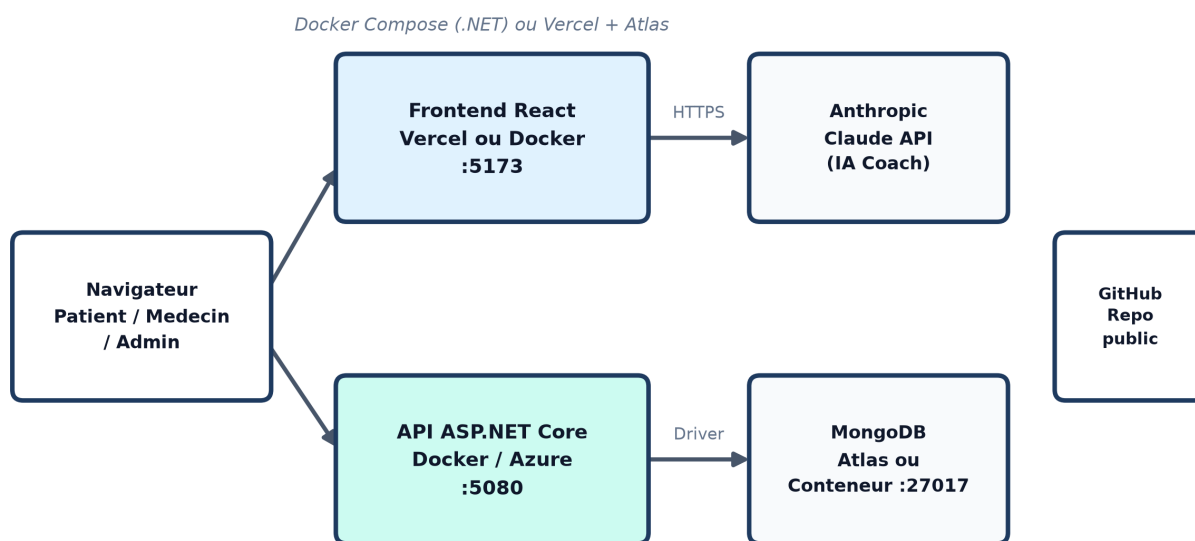


Figure 2.2 - Deployment

7.2 Variables d'environnement

- MONGODB_URI, JWT_SECRET, AES_SECRET_KEY, CLIENT_URL, ANTHROPIC_API_KEY

7.3 Cloud

Frontend Vercel, API Docker/Azure, MongoDB Atlas. Secrets hors Git.

8. Tests et comptes demo

Role	Email	Mot de passe
Admin	admin@myheath.app	Admin123
Medecin	doctor@myheath.app	Doctor123
Patient	patient@myheath.app	Patient123

8.1 Scenarios

- Admin abonnements · Patient suivi/RDV · Chat chiffre · AI Coach

9. Comparaison Node.js vs ASP.NET Core

Critere	Node.js	ASP.NET Core
Langage	JavaScript	C#
Docs	Manuelle	Swagger
PFE	MERN	Stack .NET enseignee

10. Securite

- HTTPS, secrets env, CORS, RBAC, AES, disclaimer IA

Alignement bonnes pratiques Loi 09-08 / RGPD.

11. Conclusion

Ce PFE demontre une API telemedecine ASP.NET Core 8 realisee par Nezha Fekoussa, avec React, MongoDB, Docker et deploiement cloud. Perspectives: SignalR, paiements, mobile.

11.1 Livrables

- Code GitHub: <https://github.com/salahEddine-Admou/MyHeath>
- GUIDE_INSTALLATION.md · docs/RAPPORT_PFE_MYHEATH_DOTNET.pdf
- Realisatrice: Nezha Fekoussa

Annexe A - Requetes

```
POST /api/auth/login
```

```
{ "email": "patient@myheath.app", "password": "Patient123" }
```

Annexe B - Checklist Docker

- Docker Desktop · clone · compose up --build · healthcheck · login

Annexe C - Glossaire

- JWT: JSON Web Token
- AES: Chiffrement symetrique
- RBAC: Controle d'accès par rôles
- Swagger: Documentation OpenAPI